

Usare Claude per costruire un'app

Quello che ho imparato costruendo «Ricettario» — errori compresi

10/09/2026

1. Con Claude gratuito si riesce?

L'app sì, il modo di lavorare no. Sono due cose diverse che si confondono facilmente.

Claude Code — quello che legge e scrive i tuoi file, esegue script, apre il browser e verifica — **non è nel piano gratuito**: serve un abbonamento. I termini cambiano, vanno verificati al momento.

Claude in chat, quello gratuito, può comunque scrivere tutto il codice di un'app come questa. Cambia il modo: il codice te lo scrive lui, ma **i file li componi tu, le prove le fai tu** e gli riporti il risultato.

Per un'app di questa forma la chat se la cava meglio di quanto sembri, perché è un solo HTML più due file JavaScript. Con gli **Artifact** la chat mostra l'anteprima dal vivo di un file HTML autonomo: vedi subito cosa stai costruendo.

Dove la chat gratuita fatica davvero

- **Gli asset.** Estrarre le immagini da un PDF, disegnare icone, generare grafica: sono script da eseguire. Claude te li scrive, ma li lanci tu e gli riporti com'è venuto — e sulle icone ci vogliono tre o quattro giri a testa.
- **La lunghezza.** Un progetto così sono molte ore. Sul piano gratuito i limiti d'uso ti fermano spesso, e ogni conversazione nuova riparte senza memoria: devi reincollare il codice ogni volta.
- **La verifica.** È il punto dove si sbaglia di più, anche avendo tutti gli strumenti.

In pratica: con la chat gratuita arrivi a un'app funzionante e bella. Ci metti più tempo, fai tu il lavoro di braccia, e rinunci alle rifiniture che richiedono di generare immagini.

2. Chat o Code?

Dipende dalla fase, non dai gusti.

Comincia in chat, anche se hai l'abbonamento. Per decidere l'architettura, farti spiegare le alternative, scrivere lo schema del database e ottenere una prima versione in un file solo, la chat è più veloce — e ti costringe a capire cosa stai facendo, perché il codice ti passa davanti agli occhi.

Passa a Code quando i file diventano più di due o tre, quando cominci a iterare sulla grafica, o quando la domanda diventa «ma funziona davvero?». Da lì in poi il valore non è più scrivere codice: è **provarlo**.

3. «Un solo file HTML» non vuol dire senza JavaScript

Un file `.html` può contenere tutti e tre i linguaggi: il markup, il CSS dentro `<style>` e il JavaScript

dentro `<script>`. Il browser non vede differenza fra uno `<script>` nella pagina e un file `.js` a parte.

Conviene tenerli insieme all'inizio perché l'anteprima dal vivo della chat funziona con un file solo: se il JavaScript sta fuori, l'anteprima non lo trova e vedi una pagina morta. **Conviene separarli dopo**, perché un file unico che cresce diventa ingestibile e ogni modifica ti costringe a reincollare tutto.

Nel Ricettario la divisione è questa: **HTML e CSS insieme** in `index.html`, perché struttura e stile si toccano nello stesso momento; **la logica** in `app.js`, che è la parte che cresce di più; **la rete** isolata in `supabase.js`, così il resto dell'app non sa nulla di HTTP.

4. Prima di iniziare

Porta un'immagine, non una descrizione

La cosa che ha funzionato meglio in questo progetto è stato il primo messaggio: un PDF con il rendering di come doveva venire. Quel file valeva dieci paragrafi di spiegazione — ne sono usciti i colori, i caratteri, l'impaginazione e persino le icone. Se non sai disegnare va bene una foto, lo screenshot di un'app che ti piace, uno schizzo su carta.

Poni i vincoli, non le soluzioni

«Usa le stesse tecnologie dell'altra app» ha eliminato in un colpo una decina di decisioni. Vincoli utili: quali servizi vuoi usare, quante persone la useranno, se hai già un database, quanto vuoi spendere.

Chiedi che ti faccia domande

Prima che scriva una riga, fatti elencare i dubbi. Su questo progetto le domande iniziali sono state quattro e sono servite tutte: senza, il modello di accesso sarebbe stato indovinato e da rifare.

5. Mentre costruisci

Un pezzo alla volta, e ognuno provato

Non «fammi l'app», ma «fammi l'indice», poi «fammi il dettaglio», poi «aggiungi il salvataggio». Dopo ogni pezzo, provalo davvero.

Chiedi sempre: «come faccio a verificare che funzioni?»

È la domanda più redditizia di tutte. A volte la risposta è un comando da lanciare — per esempio quello che controlla che senza login il database non restituisca niente.

Distingui «fatto» da «verificato». È l'errore che ho commesso tre volte di seguito su un problema di grafica: ho detto «risolto» quando avevo soltanto *ipotizzato* una causa. Chiedi esplicitamente: «**l'hai verificato o è un'ipotesi?**» e «**cosa non hai potuto provare?**». Un assistente onesto ti dice dove non arriva — e se non te lo dice spontaneamente, chiediglielo.

Le tue schermate valgono più delle tue parole

Il problema più lungo di questo progetto si è sbloccato quando sono arrivate le foto dello schermo:

da lì si poteva **misurare il colore dei pixel** e risalire alla causa, invece di ragionare a vuoto. Quando qualcosa non va: fotografa e manda.

Racconta il sintomo, non la diagnosi

«Su un altro iPhone la banda sta in basso» è stata l'informazione decisiva: una banda che cambia posizione da un telefono all'altro non può essere la barra di stato. Descrivi cosa vedi, dove e quando; l'interpretazione lasciala all'assistente.

6. Sulle cose noiose

Fatti scrivere anche quello che non è codice

Le istruzioni, le modifiche al database numerate, i commenti che spiegano *perché* una riga è lì. Serve a te fra sei mesi, e serve all'assistente nella prossima conversazione: se il progetto è documentato, riparte in fretta.

Le modifiche al database sempre come file separati e ripetibili

Scritte in modo che rilanciarle due volte non faccia danni. Quando lavori su dati veri, il fatto che un comando sia innocuo se ripetuto vale molto.

Il backup dei dati dal primo giorno

Il codice sta su GitHub e non lo perdi. I contenuti no: sono l'unica cosa davvero irrecuperabile.

7. Sulla sicurezza

Chiedi sempre «**questo segreto dove finisce?**». In un'app come questa una chiave sta nel JavaScript in chiaro ed è giusto così — è pubblica per definizione — mentre un'altra non deve comparire da nessuna parte. Sapere quale è quale è la differenza fra un'app sicura e una aperta a tutti.

Non fidarti del «non dà errori». In questa architettura un accesso non autorizzato risponde **200 OK con una lista vuota**, non con un errore. Sembra che funzioni. L'unico modo di sapere se i dati sono protetti è provare a leggerli senza credenziali e guardare cosa torna.

8. Come organizzare le sessioni

1. **Progetto.** Carichi il disegno, spieghi cosa fa l'app, chiedi che ti faccia domande e che proponga l'architettura. Non scrivere ancora codice.
2. **Database.** Fatti scrivere tabelle e regole di accesso, esegui, verifica che senza login non esca niente.
3. **Prima versione.** Chiedi *un solo file HTML autonomo* con dentro anche CSS e JavaScript: nell'anteprima lo vedi dal vivo. Iterate sulla grafica finché ti piace.
4. **Il collegamento al database.** Login e salvataggio. Qui separi il file della rete.
5. **Pubblicazione.** Repository, hosting, icona sul telefono.
6. **Rifiniture.** Una per conversazione, incollando ogni volta il file corrente.

Il passaggio 3 è quello dove la chat rende di più e Code di meno. Il 6 è l'opposto.

Consiglio pratico: all'inizio di ogni nuova conversazione incolla la guida all'architettura. È scritta apposta per essere letta da un assistente e gli evita di ripartire da zero — e contiene già le insidie che hai pagato tu, quindi non le ripaga lui.

9. Frasi da tenere a portata di mano

- «Prima di scrivere codice, fammi le domande che ti servono.»
- «Come faccio a verificare che funzioni?»
- «L'hai verificato o è un'ipotesi?»
- «Cosa non hai potuto provare?»
- «Spiegami perché hai scelto questo invece di quell'altro.»
- «Questo segreto dove finisce?»
- «Fammi la modifica in modo che rilanciarla due volte non faccia danni.»
- «Commenta il perché, non il cosa.»
- «Cosa si rompe se questa cosa cresce di dieci volte?»

10. In sintesi

- La chat gratuita basta per costruirla; l'abbonamento serve per farsela verificare.
- Comincia in chat, passa a Code quando i file si moltiplicano.
- Un'immagine all'inizio vale più di mille parole di specifica.
- Chiedi le domande prima del codice.
- Un pezzo alla volta, ognuno provato.
- «Fatto» e «verificato» non sono la stessa cosa: chiedi quale dei due.
- Manda le schermate, descrivi i sintomi, lascia a lui la diagnosi.
- I dati sono l'unica cosa che non torna indietro: pensa al backup subito.